



Universidad Nacional de Asunción
Facultad Politécnica
Ingeniería en Informática
Propuesta de Proyecto Final de Carrera

Propuesta de un proceso integrado de aseguramiento
de calidad (QA) en entornos de despliegue
automatizado (DevOps)
Enrique G. Sánchez Caballero

TUTOR: Prof. Ing. Ellen Lujan Mendez Xavier, Mag.

2025

1. SÍNTESIS DEL PROYECTO

1.1 TÍTULO INICIAL DEL PROYECTO

Propuesta de un proceso integrado de aseguramiento de calidad (QA) en entornos de despliegue automatizado (DevOps)

SINTETIZAR EL PROYECTO EN TRES PALABRAS CLAVE

Automatización

Métricas

Pipeline CI/CD

1.2 DURACIÓN DEL PROYECTO

12 MESES

1.3 DATOS DE LOS ESTUDIANTES Y DEL ASESOR DEL PROYECTO

NOMBRE DEL ASESOR: ELLEN LUJAN MENDEZ XAVIER	
Email: emendez@pol.una.py	Teléfono: 595991805937
CARGO: Profesor Orientador	FIRMA:

NOMBRE DEL ALUMNO: Enrique Gabriel Sánchez Caballero	
CI: 5788077	PLAN DE ESTUDIOS: XX
Email: enriquecaballero01@fpuna.edu.py	Teléfono: +595 986 129386
AÑO DE INGRESO: 2017	FIRMA:

RESUMEN DEL PROYECTO

El desarrollo moderno de software requiere de procesos rigurosos de Aseguramiento de Calidad (QA) para garantizar que los productos desarrollados cumplan con los requerimientos establecidos, se minimicen los defectos y se mantenga una calidad alta durante todo el ciclo de desarrollo del software. Según el ISTQB (International Software Testing Qualifications Board), el QA es esencial no solo para encontrar bugs sino entender como el testing reduce riesgos y valida que el software cumpla con las necesidades de los usuarios y del negocio. Se destaca que el testing debe iniciar en épocas tempranas y debe ser establecido continuamente en todas las fases del desarrollo, detectando errores lo antes posible, lo cual ayuda a ahorrar recursos y aumentar la confianza en el producto. [1]

DevOps es un enfoque de Desarrollo de software que integra a los equipos de Desarrollo y Operaciones en una entidad unificada para optimizar la entrega de software y mejorar la colaboración. Fomenta la integración y entrega continua y especialmente la automatización a lo largo de todo el ciclo de vida del producto, lo cual alivia las limitaciones de los métodos tradicionales y hasta de los ágiles. Uno de los aspectos más importantes de esta práctica es la implementación del CI/CD (Integración Continua/Entrega Continua), que automatiza el compilamiento, el testeo y el desarrollo del software. Esto permite que los cambios hechos en el código se integren continuamente y se prueben automáticamente, permitiendo lanzamientos rápidos y confiables con una mínima intervención manual. [2]

La integración de procesos de QA dentro de entornos DevOps y pipelines CI/CD representa el siguiente paso hacia un proceso de entrega de software más confiable, ágil y eficiente. Al incorporar prácticas de aseguramiento de calidad desde las primeras etapas y automatizar pruebas dentro del pipeline, se garantiza que cada cambio pase por validaciones continuamente antes de ser lanzado. Esta combinación no solo acelera la detección de defectos, sino que asegura también que la calidad no se vea afectada por la velocidad de entrega. En el marco de este trabajo, se busca demostrar cómo la integración estratégica de QA en flujos DevOps, apoyado en CI/CD, puede reducir riesgos, optimizar recursos y entregar productos de alta calidad de forma consistente, alineando así los objetivos técnicos con las necesidades del negocio y la satisfacción del usuario final.

1.4 OBJETIVOS DEL PROYECTO.

1.4.1 OBJETIVO GENERAL

Se propone implementar un proceso de aseguramiento de calidad en un proyecto con el objetivo de garantizar la detección temprana de defectos. Este proceso incluye el análisis de requerimientos y del código para identificar posibles fallos desde las primeras etapas, el diseño y la ejecución de pruebas automáticas, y finalmente, la implementación de métricas que permitan identificar cuellos de botella, áreas de riesgo y oportunidades de mejora.

1.4.2 OBJETIVOS ESPECÍFICOS

1. OBJETIVO 1

Analizar los requerimientos del sistema para identificar posibles riesgos y defectos en las etapas tempranas de desarrollo.

2. OBJETIVO 2

Evaluar el código fuente a través de revisión estática y análisis automatizado para detectar errores tempranamente.

3. OBJETIVO 3

Diseñar y desarrollar un conjunto de pruebas automatizadas que cubran la funcionalidad crítica del sistema.

4. OBJETIVO 4

Integrar pruebas automáticas en el pipeline CI/CD, asegurando su ejecución continua con cada cambio introducido en el código.

5. OBJETIVO 5

Definir e implementar métricas que permitan medir la eficiencia del proceso de QA y la estabilidad del producto.

6. OBJETIVO 6

Analizar los resultados obtenidos para identificar cuellos de botella, áreas de riesgo y oportunidades de mejora continua en el flujo de trabajo.

2. FORMULACIÓN GENERAL DEL PROYECTO

2.1 PROBLEMA E HIPÓTESIS

2.1.1 ¿CUÁL ES EL PROBLEMA U OPORTUNIDAD?

En el desarrollo moderno de software actual, existe la necesidad de lanzar software rápidamente sin comprometer la calidad de este. Vivimos en una sociedad donde el tiempo es dinero, y el éxito de un producto puede verse afectada por la reputación que las personas tienen sobre este. Es por eso por lo que es absolutamente necesario evitar que bugs aparezcan en el entorno de producción, donde los usuarios finales tienen acceso a este. Los bugs pueden tener un impacto leve, como por ejemplo una experiencia de usuario afectada, a un impacto gravísimo, donde se pierden no solo cantidades enormes de dinero tanto para la empresa como para los usuarios, sino también algo hasta más importante, reputación. [3]

Especialmente en Paraguay, las empresas tienden a subestimar el impacto del proceso de Aseguramiento de Calidad. Debido a esto, prácticas de QA están muy poco desarrolladas. Esto genera que muchos proyectos prioricen la entrega rápida sobre la calidad, lo que aumenta la probabilidad de que defectos y bugs lleguen a producción.

2.1.2 ¿CUÁL ES LA SOLUCIÓN PROPUESTA POR EL PROYECTO DE INVESTIGACIÓN?

La solución que se propone consiste en implementar un proceso integral de Aseguramiento de Calidad (QA) dentro de un entorno DevOps mediante la integración de pruebas automatizadas en un pipeline CI/CD.

La propuesta se basa en la integración de pruebas automatizadas dentro de un pipeline CI/CD (Integración y Entrega Continua), permitiendo que cada cambio realizado en el código sea validado automáticamente antes de su despliegue. De esta forma, se busca prevenir defectos en lugar de detectarlos tardíamente, alineando la calidad con la agilidad del desarrollo moderno.

2.1.3 ¿CUÁL ES LA HIPÓTESIS DEL PROYECTO?

Se espera que la implementación de los procesos de Aseguramiento de Calidad en el entorno de DevOps permita capturar bugs y defectos tempranamente, reduciendo el impacto que estos podrían tener, ya que podrán ser corregidos con rapidez. Otro beneficio esperado es el aumento de la confianza al introducir cambios, considerando que la aplicación será probada automáticamente cada vez que se modifique el código. Todos estos cambios e implementaciones, tanto técnicos como culturales, buscan demostrar que los procesos de QA son esenciales y beneficiosos, y que el esfuerzo inicial requerido para adoptar esta nueva cultura se traduce en una mayor eficiencia, estabilidad y calidad del producto final.

2.2 ANTECEDENTES

El impacto del testeo de software ha sido ampliamente investigado. En [4] analizan el impacto de los lanzamientos rápidos y la relación de este con el testeo del software. El lanzamiento rápido es clave en la cultura DevOps, ya que este se basa en constantes lanzamientos para obtener retroalimentación temprana y arreglar los problemas que puedan aparecer cuanto antes. Encontraron que los Lanzamientos rápidos no tienen impacto en la calidad del producto. Cada lanzamiento tiene un alcance limitado y pequeño, lo cual fuerza a que el equipo de testers se concentre en áreas específicas, permitiendo pruebas profundas y minuciosas en estas áreas.

En [5] encontraron que los lanzamientos rápidos no necesariamente permiten lanzar contenido nuevo a los usuarios finales, sino que los beneficios principales de estos son una mayor estabilidad en el software y retroalimentación por parte de los usuarios.

En [6] investigaron cuales son los mayores cuellos de botella que existen en el proceso de lanzamiento de software. Recomendán integrar lo que ellos llaman como “Continuous testing”. Consiste en automatizar lo más posible las pruebas unitarias y de regresión, cuidando que estas se corran en un tiempo razonable.

2.3 METODOLOGÍA

La metodología para seguir se basa en la investigación científica experimental.

1. Revisión bibliográfica con el objetivo de identificar los procedimientos y herramientas comúnmente utilizadas.
2. Análisis del contexto y diagnóstico inicial, utilizándolo de base para medir los resultados obtenidos posteriormente
3. Diseño del proceso de aseguramiento de calidad a implementar, adaptando las buenas prácticas encontradas a las necesidades y limitaciones del contexto del proyecto.
4. Implementación y experimentación, donde se recolectan métricas y resultados cuantitativos para evaluar el impacto del proceso implementado.
5. Evaluación empírica de resultados

6. Conclusiones y trabajos futuros

2.4 RESULTADOS ESPERADOS DEL PROYECTO

Se espera tener un impacto positivo en el proyecto de software elegido, potencialmente detectando defectos antes de lanzamientos, aumentando la candencia de estos y la confianza de implementación de cambios en el sistema. Logrando así un entorno automatizado y con la posibilidad de expandir y mantener a medida que el proyecto vaya creciendo.

3. PLANEAMIENTO DEL PROYECTO

Actividad	Inicio	Fin	Duración en días

4. TEMARIO TENTATIVO DEL REPORTE TÉCNICO FINAL

- **Introducción**
- **Antecedentes**
- **Problemática**
- **Propuesta**
- **Experimentos y resultados**
- **Conclusiones**

REFERENCIAS BIBLIOGRÁFICAS

1. International Software Testing Qualifications Board (ISTQB). Certified Tester Foundation Level Syllabus, version 4.0 [Internet]. 2024 Nov [cited 2025 Aug 11]. Available from: https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf
2. Gokarnaa M, Singhb R. DevOps: A Historical Review and Future Works [Internet]. arXiv preprint arXiv:2012.06145; 2020 [cited 2025 Aug 12]. Available from: <https://arxiv.org/abs/2012.06145>
3. Khomh F, Adams B, Dhaliwal T, Zou Y. Understanding the impact of rapid releases on software quality: The case of Firefox. Empir Software Eng. 2014; Early view. doi:10.1007/s10664-014-9308-x
4. Mantyla, M. V., Khomh, F., Adams, B., Engstrom, E., & Petersen, K. (2013). On Rapid Releases and Software Testing. IEEE International Conference on Software Maintenance, 312, 312-322. DOI: 10.1109/ICSM.2013.13.
5. Costa, D., McIntosh, S., Treude, C. et al. The impact of rapid release cycles on the integration delay of fixed issues. Empir Software Eng 23, 835–904 (2018). <https://doi.org/10.1007/s10664-017-9548-7>
6. Kerzazi, N., & Khomh, F. (2014). Factors Impacting Rapid Releases: An Industrial Case Study. In Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '14), Torino, Italy, September 18-19, 2014 (pp. 180-190). ACM. <https://doi.org/10.1145/2652524.2652589>